

DocSphere System Architecture & Spec

Unified Multi-Source Documentation Aggregator & Progress Tracker

Document ID: SPEC-DOC-2026-V1 Author: Systems Architecture Team Stack: React, TS, Tailwind, Dexie, Jina Date: August 2026

1. Executive Summary & Core Objectives

DocSphere is a high-performance, developer-centric documentation hub designed to aggregate, standardize, and track progress across disparate technical documentation portals. Developers frequently suffer from "tab explosion" when working with complex modern stacks (e.g., React, Tailwind, PostgreSQL, Rust, AWS). Standard browsers fail to retain scroll progress, lack unified cross-doc search, and clutter reading space with redundant navigation bars and site headers.

DocSphere solves these pain points by offering a client-first, single-pane application that ingests external documentation via high-efficiency CORS proxies, converts raw HTML into distraction-free Markdown, and enforces bidirectional progress synchronization (continuous scroll monitoring + explicit section checkboxes).

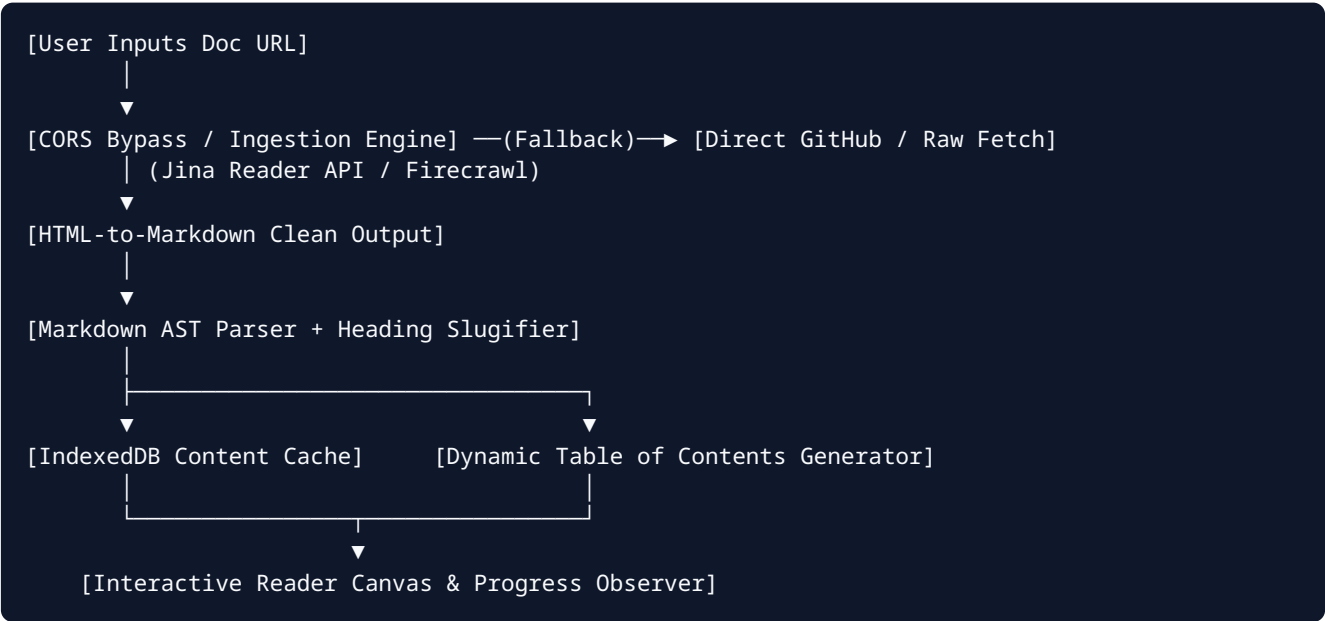
Core Design Principles

- **Tab Elimination:** Centralize all library, framework, and tool documentations inside one persistent workspace.
- **Zero-Friction Ingestion:** Ingest any doc URL with automatic layout stripping and clean Markdown extraction.
- **Granular Progress Tracking:** Math-driven scroll percentage persistence combined with section-level completion states.
- **Offline-First Persistence:** Local storage via IndexedDB for zero latency, instant retrieval, and offline reading.

2. System Architecture & Content Pipeline

The platform uses a client-driven decoupled architecture. External documentation pages are fetched via an automated proxy pipeline, parsed into AST (Abstract Syntax Tree), cached locally, and rendered with custom interactive typography.

Ingestion & Processing Pipeline



2.1 Tech Stack Specification

Layer	Technology / Library	Rationale & Role
Frontend Framework	React 18 + Vite + TypeScript	Lightning-fast build tooling, type-safe data modeling, reactive DOM updates.
Styling & Design	Tailwind CSS + @tailwindcss/typography	Utility-first CSS with optimized prose classes for technical doc rendering.
Ingestion Engine	Jina Reader API (r.jina.ai) / Firecrawl API	Bypasses client-side CORS restriction and strips headers, footers, and scripts into clean GFM.
Data Persistence	IndexedDB (via Dexie.js) + localStorage	IndexedDB stores multi-megabyte markdown content; LocalStorage stores UI state/preferences.
Markdown Engine	react-markdown + remark-gfm + rehype-highlight	Full GitHub-Flavored Markdown support, code block syntax highlighting, and custom DOM elements.
State & UI Icons	Zustand / React Context + lucide-react	Lightweight global state management for active docs, navigation tree, and iconography.

3. Data Models & Database Schema

Data is structured using TypeScript interfaces and stored in an IndexedDB schema managed by Dexie.js for asynchronous, transactional access.

```
// Data Schema Definitions (types.ts)

export interface DocSource {
  id: string;           // UUID v4
  title: string;        // User-defined or auto-parsed title
  url: string;          // Original documentation URL
  category: string;     // e.g., "Frontend", "Database", "DevOps"
  tags: string[];       // Custom user tags
  content: string;      // Raw converted Markdown text
  toc: ToCItem[];       // Parsed Table of Contents hierarchy

  // Progress Metrics
  scrollProgress: number; // Float 0.0 to 100.0 (percentage)
  completedSections: string[]; // Array of completed heading slug IDs
  isCompleted: boolean;  // Overall doc completion status

  // Timestamps
  addedAt: number;       // Timestamp (ms)
  lastReadAt: number;    // Timestamp (ms)
}

export interface ToCItem {
  id: string;           // Heading slug ID (e.g., "installation-guide")
  text: string;         // Display heading title
  level: number;        // Heading depth (1 for H1, 2 for H2, 3 for H3)
}

export interface WorkspaceSettings {
  theme: 'dark' | 'light' | 'system';
  autoScrollToLastPosition: boolean;
  fontSize: number;      // Reading font size in px
  sidebarCollapsed: boolean;
}
```

4. Core Functional Modules & Implementation Details

4.1 Ingestion Service (`docFetcher.ts`)

Standard browser `fetch()` calls fail against 95% of documentation sites due to cross-origin policies (CORS). DocSphere implements a tiered proxy strategy:

- **Tier 1 Proxy (Jina Reader):** Prepends `https://r.jina.ai/` to target URL. Converts HTML to clean Markdown serverlessly.
- **Tier 2 Proxy (GitHub Raw Handler):** Detects GitHub URLs (e.g., `github.com/user/repo/blob/main/README.md`) and automatically transforms them to raw content endpoints (`raw.githubusercontent.com`).
- **Tier 3 Fallback (CorsAnywhere / Custom Proxy):** If proxy fails, routes request through a fallback CORS proxy for raw HTML extraction via `TurndownService`.

4.2 Dual-Engine Progress Tracking

DocSphere combines **continuous scroll position tracking** with **discrete section checkboxes** to give developers total visibility over their reading progress.

1. Continuous Scroll Observer

Tracks viewport percentage within the scrollable container:

```
Progress % = (ScrollTop / (ScrollHeight - ClientHeight)) * 100
```

Updates are debounced by 300ms to avoid unnecessary IndexedDB write load.

2. Discrete Heading Checkbox State

Injects interactive checkboxes into all `H1`, `H2`, and `H3` elements rendered by `react-markdown`.

Checking off sections recalculates global doc progress and updates the interactive Table of Contents drawer.

4.3 Dynamic Table of Contents (ToC) & ScrollSpy

When Markdown content is loaded, a regex-based AST scanner extracts heading nodes (`#`, `##`, `###`) to construct a structured outline tree. An `IntersectionObserver` targets all heading IDs in the DOM, highlighting the current heading in the ToC sidebar as the user scrolls.

5. UI Layout Blueprint & Keyboard Navigation

5.1 Three-Pane Workspace Wireframe

DOCSPHERE NAV	HEADER: React Docs (Progress: 68%) [Complete]	OUTLINE / TOC
[+ Add New Doc]	# Quick Start	• Quick Start
[Search Docs...]	[x] Section 1: Installation	• Installation
📁 FRONTEND	To install React, run the following:	- NPM Setup
• React (68%)	```bash	- Vite Setup
• Tailwind (100%)	npm install react react-dom	• Core Concepts
	```	- Hooks
📁 BACKEND	[ ] Section 2: Core Concepts	- Context
• Go Docs (25%)		• API Reference

## 5.2 Keyboard Shortcuts Matrix

Shortcut	Action	Context
Cmd / Ctrl + K	Open Global Command Palette (Quick Switch Docs / Search)	Global
Cmd / Ctrl + B	Toggle Navigation Sidebar	Global
j / k	Smooth Scroll Down / Up (Vim Navigation)	Reader Viewport
Cmd / Ctrl + Enter	Mark Current Active Section as Complete	Reader Viewport
Cmd / Ctrl + D	Toggle Overall Doc Completion Status	Reader Viewport

## 6. Non-Functional Requirements & Security

- **Sanitization & XSS Prevention:** Raw HTML embedded inside ingested Markdown files is sanitized through `DOMPurify` before rendering to prevent malicious script injection.
- **Storage Quota Management:** Modern browsers allocate 500MB+ for IndexedDB. Cached docs are stored as plain text string blobs (~50KB per document), allowing up to 10,000+ docs to be cached offline without performance degradation.
- **Memory Optimization:** Large documentation files (e.g., single-page monolithic API specs) use virtualization for code blocks and heavy DOM elements to maintain 60 FPS scrolling performance.

## 7. Implementation Roadmap

Phase	Target Focus	Key Deliverables	Timeline
PHASE 1	Core Ingestion & Reader	Vite setup, Tailwind typography, Jina Reader integration, ReactMarkdown viewer.	Days 1–3
PHASE 2	Progress System	Scroll observer, section completion toggles, IndexedDB (Dexie) integration.	Days 4–6
PHASE 3	Organization & ToC	Dynamic ToC drawer, ScrollSpy observer, category hierarchy, tag system.	Days 7–9
PHASE 4	Polish & Power Tools	Command palette (`Cmd+K`), JSON export/import backup, keyboard hotkeys, offline PWA.	Days 10–12